

Notizen

R für Data Science (Wickham & Grolemund 2018/2024)

2026-05-14

Table of contents

1 Data Science: Überblick	1
2 Grundlagen des Programmierens in R	2
2.1 Operatoren in R	2
3 Vektoren & Datenstrukturen	3
3.1 Atomare Vektoren	4
3.2 Listen (rekursive Vektoren)	6
3.3 Erweiterter Vektoren	7
4 Daten importieren	8
4.1 CSV mit readr	8
5 Datentransformation mit dplyr	9
6 Fehlende Werte (NA)	9
7 Explorative Datenanalyse (EDA)	10
7.1 Variation einer kategorialen Variable	10
7.2 Variation einer kontinuierlichen Variable	11
7.3 Co-Variation (zwei Variablen)	12
8 ERGÄNTZEN UM DIE GGLOT DARSTELLUNGEN!	13

1 Data Science: Überblick

Rohdaten → Verständnis → Erkenntnis → Wissen

Der Data-Science-Workflow ist iterativ:

Importieren → **Bereinigen/Aufbereiten** → **Transformieren** (Daten filtern, neue Variablen bilden, aggregieren, reduzieren) → **ggf. Modellieren** → **Visualisieren** → **Kommunizieren**

Die Grundlage für all das: **Programmieren** (hier: R).

2 Grundlagen des Programmierens in R

- R als Taschenrechner
- Objektnamen in `snake_case`
- R ist pingelig: Groß-/Kleinschreibung, Tippfehler etc. führen zu Fehlern
- Klassen: `int`, `dbl`, `chr`, `dtm`, `lgl`, `fct`, `date`

2.1 Operatoren in R

2.1.1 Rechenoperatoren

Operator	Bedeutung	Beispiel	Ergebnis
+	Addition	3 + 2	5
-	Subtraktion	5 - 2	3
*	Multiplikation	4 * 3	12
/	Division	7 / 2	3.5
^	Potenz	2^3	8
%%	Modulo (Rest)	7 %% 2	1
%/%	Ganzzahldivision	7 %/% 2	3

```
2 + 3      # 5
10 / 3     # 3.333...
10 %% 3    # 1 (Rest bei Division)
10 %/% 3   # 3 (ganzzahliger Anteil)
2^8        # 256
```

2.1.2 Zuweisung

```
x <- 42     # Standard-Zuweisung in R
x           # gibt 42 aus
```

2.1.3 Logische Operatoren

Operator	Bedeutung
&	und (vektoriert)
	oder (vektoriert)
!	nicht
%in%	ist Element von

```
TRUE & FALSE    # FALSE
TRUE | FALSE    # TRUE
!TRUE           # FALSE

x <- c(1, 2, 3, 4, 5)
x %in% c(2, 4)  # FALSE TRUE FALSE TRUE FALSE
```

2.1.4 Vergleichsoperatoren

Operator	Bedeutung
>	größer als
>=	größer oder gleich
<	kleiner als
<=	kleiner oder gleich
!=	ungleich
==	gleich

Achtung Fließkomma: Bei Vergleichen mit == aufpassen — Ungenauigkeiten durch endliche Nachkommastellen in dbl. Besser `dplyr::near()` verwenden:

```
0.1 + 0.2 == 0.3    # FALSE (!)
dplyr::near(0.1 + 0.2, 0.3) # TRUE
```

Spezialwerte bei Division:

```
c(-1, 0, 1) / 0    # -Inf  NaN  Inf
```

Für Prüfungen auf diese Sonderfälle **nicht** == verwenden, sondern:

Funktion	erkennt 0	erkennt Inf	erkennt NA	erkennt NaN
<code>is.finite()</code>	x			
<code>is.infinite()</code>		x		
<code>is.na()</code>			x	x
<code>is.nan()</code>				x

3 Vektoren & Datenstrukturen

- Vektoren sind die Objekte, auf denen Dataframes aufsetzen.
- beispielsweise ist klassischerweise jede Spalte (Variable) ein Vektor

- es gibt verschiedene Vektortypen: atomare Vektoren (logical, integer, double, character), Faktoren (kategoriale Daten) und Listen (rekursive Vektoren)

Sonderform: NULL

NULL ist ein Objekt, das das **Fehlen eines Vektors** darstellt; es verhält sich wie ein Vektor der Länge 0.

```
length(NULL) # 0
is.null(NULL) # TRUE
```

Zwei wichtige Eigenschaften jedes Vektors:

```
typeof(x) # Typ
length(x) # Länge
```

3.1 Atomare Vektoren

Atomare Vektoren sind immer **homogen** (alle Elemente gleichen Typs). Es gibt fünf Typen:

Typ	Kurzform	Beispiel	Hinweis
logical	lgl	TRUE, FALSE, NA	intern als 0/1 gespeichert
integer	int	1L, 42L	ganze Zahlen; NA_integer_
double	dbl	3.14, 1e5	Gleitkommazahlen; NA_real_, NaN, Inf
character	chr	"SPD", "Berlin"	Strings; NA_character_
raw	—	—	für Datenanalyse kaum gebräuchlich
complex	—	1+2i	für Datenanalyse kaum gebräuchlich

Typ prüfen mit `typeof()` oder den `is_*`-Funktionen aus `purrr` (zuverlässiger als `is.vector()` oder `is.atomic()`):

Funktion	lgl	int	dbl	chr	list
<code>is_logical()</code>	x				
<code>is_integer()</code>		x			
<code>is_double()</code>			x		
<code>is_numeric()</code>		x	x		
<code>is_character()</code>				x	
<code>is_atomic()</code>	x	x	x	x	
<code>is_list()</code>					x
<code>is_vector()</code>	x	x	x	x	x

3.1.1 Vektoren konvertieren

Explizite, lokale Typumwandlung mit `as.*`:

```
as.integer(3.7)    # 3  (schneidet ab, rundet nicht)
as.double(5L)      # 5
as.character(42)   # "42"
as.logical(0)      # FALSE; 0 = FALSE, alles andere = TRUE
```

Vorsicht: R macht das auch **implizit**, wenn der Typ nicht zur Rechenoperation passt. Das kann zu stillen Fehlern führen.

3.1.1.1 Zeichenvektoren (charakter) als eine Abfolge von Strings

Ein Zeichenvektor ist der komplexeste atomare Typ. Jedes Element ist ein String; ein String kann beliebig viel Text enthalten. R verwendet intern einen **globalen Stringpool** — jeder eindeutige String wird nur einmal im Speicher abgelegt, was Speicher spart.

```
# Mehrere Strings in einen Zeichenvektor stecken
parteien <- c("SPD", "CDU", "Grüne", "FDP", "AfD")
```

Für “literale” Anführungszeichen innerhalb eines Strings: Escape-Zeichenfolge mit `\`:

```
x <- "Sie sagte: \"Hallo\""    # \" für Anführungszeichen
y <- 'Es war\'s wert'          # \' für Apostroph
z <- "Pfad: C:\\Users\\name"    # \\ für einen Backslash
```

Weitere Sonderzeichen:

- `\n` — neue Zeile
- `\t` — Tabulator
- `\uXXXX` — Unicode-Zeichen (z.B. `\u00e4` = ä)

Die angezeigte Darstellung eines Strings (`print()`) ist nicht zwingend gleich dem Stringinhalt. Um den unformatierten Inhalt zu sehen: `writeLines()`

```
x <- "Zeile 1\nZeile 2"
print(x)      # "Zeile 1\nZeile 2"  (mit \n sichtbar)
writeLines(x) # Zeile 1
              # Zeile 2
```

Alle `stringr`-Funktionen beginnen mit `str_`:

```
stringr::str_length(parteien)      # Anzahl Zeichen
stringr::str_to_upper(parteien)    # Großschreibung
stringr::str_detect(parteien, "D") # enthält "D"?
stringr::str_replace(parteien, "ü", "ue") # ersetzen
stringr::str_c("Partei: ", parteien) # zusammenfügen
```

3.1.1.1 Strings und stringr

```
# Mehrere Strings in einen Zeichenvektor
parteien <- c("SPD", "CDU", "Grüne")

# Alle stringr-Funktionen beginnen mit str_
stringr::str_length(parteien)      # Anzahl Zeichen je Element
stringr::str_to_lower(parteien)    # kleinschreiben
stringr::str_to_upper(parteien)    # GROSSschreiben
stringr::str_detect(parteien, "ü") # logischer Vektor
stringr::str_replace(parteien, "ü", "ue") # erstes Vorkommen
stringr::str_replace_all(parteien, "S", "s") # alle Vorkommen
stringr::str_c(parteien, collapse = ", ") # zusammenkleben
```

3.2 Listen (rekursive Vektoren)

Listen können **heterogen** sein — Elemente dürfen unterschiedliche Typen und Längen haben. Listen können sogar andere Listen enthalten.

```
# Liste erstellen
meine_liste <- list(1, "a", TRUE, c(1, 2, 3))

# Überblick
str(meine_liste) # str() ist sinnvoll für einen Überblick
```

Listen auch andere Listen aufnehmen:

```
z <- list(list(1, 2), list(3, 4))
```

Listen visualisieren (vereinfachtes Schema):

```
x1 <- list(1, 2, 3, 4)      # 4 Elemente, jedes Länge 1
x2 <- list(list(1, 2), list(3, 4)) # 2 Elemente, je Länge 2
x3 <- list(1, list(2, list(3))) # verschachtelt
```

Listen sind sinnvoll für **verschachtelte Daten** (z.B. JSON-Importe, Modell-Outputs).

3.3 Erweiterter Vektoren

Vektoren können zusätzliche **Attribute** enthalten, die sie zu erweiterten Vektoren machen:

- **Faktoren** — erweiterte integer-Vektoren (Attribut: `levels`)
- **Datums- & Zeitwerte** — erweiterte numerische Vektoren
- **Dataframes & Tibbles** — erweiterte Formen von Listen

3.3.1 Faktoren (forcats)

Faktoren (`fct`) dienen zur Darstellung **kategorialer Daten**.

- Bauen auf Ganzzahlen (integer) auf und besitzen ein zusätzliches `levels`-Attribut
- Liste der gültigen Stufen = **Levels**
- ggplot2 und andere tidyverse-Funktionen nutzen die Stufenreihenfolge für Achsen, Legenden und Farben — und halten sie ein
- Beides (integer-Codes und Levels) mit `attributes()` anzeigen

```
# Faktor erstellen
region <- factor(c("Nord", "Süd", "Ost", "West", "Nord"),
                 levels = c("Nord", "Süd", "Ost", "West"))

levels(region)      # "Nord" "Süd" "Ost" "West"
attributes(region) # levels + class
```

Kategoriale Variablen sind i.d.R. als **Faktoren** oder **Zeichenvektoren** gespeichert.

3.3.1.1 Stufenreihenfolge ändern (forcats)

```
# Nach einer numerischen Variable sortieren (aufsteigend)
df$partei <- forcats::fct_reorder(df$partei, df$stimmenanteil)

# Nach zunehmender Häufigkeit sortieren (gut für Balkendiagramme)
df$partei <- forcats::fct_infreq(df$partei)

# Reihenfolge umkehren
df$partei <- forcats::fct_rev(df$partei)

# Tipp: fct_reorder2() für Liniendiagramme -
# passt Linien an die Legendenreihenfolge an
df$partei <- forcats::fct_reorder2(df$partei, df$jahr, df$stimmenanteil)
```

3.3.1.2 Faktorstufen selbst ändern

```
# fct_recode(): Syntax ähnlich rename()
# Nicht explizit geänderte Stufen bleiben unverändert.
# Mit fct_recode() lassen sich auch Stufen zusammenlegen.
df$partei <- forcats::fct_recode(df$partei,
  "CDU/CSU" = "CDU",
  "CDU/CSU" = "CSU",      # CSU wird mit CDU zusammengelegt
  "Sonstige" = "FW",
  "Sonstige" = "SSW"
)
```

4 Daten importieren

4.1 CSV mit readr

```
df <- readr::read_csv(
  here::here("data", "meine_daten.csv"),
  comment = "#",      # Zeilen mit # am Anfang überspringen
  # skip   = 6,        # alternativ: n Headerzeilen überspringen
  col_names = TRUE,    # FALSE → R vergibt X1 bis Xn
  na        = ".",     # falls fehlende Werte mit "." dargestellt sind
)
```

4.1.1 Parsen

readr versucht den Typ automatisch zu erkennen — aber manchmal muss man nachhelfen, z.B. bei europäischen Zahlenformaten ("1.234,56") oder speziellen NA-Darstellungen ("-"). Parse-Funktionen konvertieren einen Zeichenvektor (character) in einen anderen Datentyp. Parsen = aus einem String einen typisierten Wert machen.

```
readr::parse_integer(c("1", "2", "3")) # "1" → 1L
readr::parse_integer(c("1", "-", "3"), na = "-") # "-" → NA
readr::parse_double("3.14") # "3.14" → 3.14
readr::parse_number("1.234,56",
  locale = readr::locale(grouping_mark = ".")) # europäisches Format
readr::parse_logical(c("TRUE", "FALSE", "true"))
readr::parse_character("München")
readr::parse_factor(c("Nord", "Süd"), levels = c("Nord", "Süd", "Ost"))
readr::parse_date("2025-09-26")
readr::parse_time("14:30:00")
readr::parse_datetime("2025-09-26T14:30:00")
```



```
# Parsing-Fehler lassen sich mit `problems()` abrufen.
x <- readr::parse_integer(c("1", "abc", "3"))
readr::problems(x)
```

5 Datentransformation mit dplyr

```
library(dplyr)

# Die fünf Kernverben (+ group_by / ungroup)
df %>%
  dplyr::filter(stimmenanteil > 5) %>%      # Zeilen filtern
  dplyr::arrange(desc(stimmenanteil)) %>%    # sortieren (absteigend)
  dplyr::select(wahlkreis, partei, stimmenanteil) %>% # Spalten auswählen
  dplyr::mutate(anteil_log = log(stimmenanteil)) %>% # neue Variable
  dplyr::summarize(mean_anteil = mean(stimmenanteil, na.rm = TRUE))

# Gruppenweise Anwendung mit group_by + ungroup
df %>%
  dplyr::group_by(partei) %>%
  dplyr::summarize(
    n          = dplyr::n(),
    mean_anteil = mean(stimmenanteil, na.rm = TRUE),
    sum_mandate = sum(mandate, na.rm = TRUE)
  ) %>%
  dplyr::ungroup()
```

6 Fehlende Werte (NA)

„Implizite Missings sind die Abwesenheit von Anwesenheit, explizite Missings die Anwesenheit von Abwesenheit.“

- NA = „not available“
- **Implizite Missings:** Abwesenheit von Anwesenheit (Zeile fehlt ganz)
- **Explizite Missings:** Anwesenheit von Abwesenheit (Zelle enthält NA)
- NAs sind „ansteckend“: fast jede Operation auf einem NA gibt NA zurück
- In aggregierenden Funktionen: `na.rm = TRUE` — schließt NAs aus, ohne zu viele auszuschließen

```

# Prüfen
is.na(x)                # logischer Vektor
sum(is.na(df$einkommen)) # Anzahl NAs
mean(is.na(df$einkommen)) # Anteil NAs

# NAs bei einer spezifischen Berechnung ignorieren
mean(df$einkommen, na.rm = TRUE)

# NAs dauerhaft ausschließen mit filter(...)
df %>%
  dplyr::filter(!is.na(einkommen_je_ew_2021))

# ...oder mit drop_na() aus tidyr
df %>%
  tidyr::drop_na(einkommen_je_ew_2021, alo_quote_insgesamt)

```

7 Explorative Datenanalyse (EDA)

EDA ist ein iterativer Prozess: Fragen stellen → Daten transformieren, visualisieren, modellieren → neue Erkenntnisse gewinnen → neue Fragen stellen.

„Lieber eine ungefähre Antwort auf die richtige Frage, als eine genaue Antwort auf die falsche.“ — John Tukey

„There are no routine statistical questions, only questionable statistical routines.“ — Sir David Cox

EDA-Fragen kreisen immer um **Variation** einer einzelnen Variable oder **Co-Variation** zweier Variablen.

Variablentyp	Rechnen	Visualisierung
Kategorial	Häufigkeitszählungen mit <code>table()</code> oder <code>count()</code>	Balkendiagramm
Metrisch / kontinuierlich	Kennwerte (mean, sd, ...)	Histogramm oder Häufigkeitspolygon; Boxplot

7.1 Variation einer kategorialen Variable

```

# Häufigkeitstabelle
df %>%
  dplyr::group_by(bundesland) %>%
  dplyr::summarize(
    n          = dplyr::n(),
    n_valid    = sum(!is.na(bundesland)),

```

```

    n_miss    = sum(is.na(bundesland))
  ) %>%
  dplyr::mutate(
    anteil_pct    = n / sum(n) * 100,
    kumuliert_pct = cumsum(anteil_pct)
  )

# Balkendiagramm
barplot(table(df$bundesland),
        las = 2,
        main = "Fälle pro Bundesland")

```

7.2 Variation einer kontinuierlichen Variable

Lagemaße

```

mean(x, na.rm = TRUE)    # Arithmetisches Mittel
median(x, na.rm = TRUE)  # Median

# Modus: kein eingebauter Befehl
names(sort(table(x), decreasing = TRUE))[1]

```

Streuungsmaße

```

sd(x, na.rm = TRUE)      # Standardabweichung
IQR(x, na.rm = TRUE)     # Interquartilsabstand
mad(x, na.rm = TRUE)     # Median Absolute Deviation

```

Extremwerte und Quantile

```

min(x, na.rm = TRUE)
max(x, na.rm = TRUE)
range(x, na.rm = TRUE)
quantile(x, probs = 0.25)
quantile(x, probs = c(.25, .75))

```

Positionsmaße

```

dplyr::first(x)
dplyr::last(x)
dplyr::nth(x, 3)

```

Visualisierung: Boxplot

Der Boxplot fasst die wichtigsten Kennwerte grafisch zusammen: Median (Linie), IQR (Box), Whisker ($1.5 \times \text{IQR}$), Ausreißer (Punkte).

```

boxplot(df$einkommen_je_ew_2021,
        main = "Einkommensverteilung",
        ylab = "Euro pro EW")

# Gruppiert nach kategorialer Variable
boxplot(einkommen_je_ew_2021 ~ bundesland,
        data = df,
        las = 2,
        main = "Einkommen nach Bundesland",
        ylab = "Euro pro EW")

```

Visualisierung: Histogramm

```

hist(df$einkommen_je_ew_2021,
     main = "Einkommensverteilung",
     xlab = "Euro pro EW")

```

7.3 Co-Variation (zwei Variablen)

Kontinuierlich × kategorial:

- Häufigkeitspolygon: Verteilung einer kontinuierlichen Variable, untergliedert nach einer kategorialen Variable
- Wenn Gruppen sehr unterschiedlich groß sind → Verteilungsunterschiede schlecht erkennbar → **standardisieren mit Dichte** (`y = ..density..`)

```

# Häufigkeitspolygon nach Gruppe (Base R)
plot(density(df$einkommen[df$ost == 1]),
     col = "steelblue",
     main = "Einkommensverteilung Ost vs. West",
     xlab = "Einkommen")
lines(density(df$einkommen[df$ost == 0]), col = "tomato")
legend("topright",
     legend = c("Ost", "West"),
     col = c("steelblue", "tomato"),
     lty = 1)

```

Zwei kategoriale Variablen:

- `geom_count()` als bildliche Darstellung einer Kreuztabelle (Punktgröße = Häufigkeit)

```
table(df$partei, df$geschlecht) # Kreuztabelle
```

Zwei kontinuierliche Variablen:

- Streudiagramm / Scatterplot: `plot()` (Base R) oder `geom_point()` + `alpha`

- Bei Overplotting: `geom_bin2d()` oder `geom_hex()` (aus `hexbin`)

```
# Scatterplot mit transparenten Punkten
plot(df$einkommen_je_ew_2021,
     df$alo_quote_insgesamt,
     xlab = "Einkommen je EW",
     ylab = "Arbeitslosenquote",
     pch = 19,
     col = rgb(0, 0, 1, 0.3)) # alpha über rgb()
```

8 ERGÄNTZEN UM DIE GGLOT DARSTELLUNGEN!